

머신러닝 3 교시

의사결정나무 (Decision Tree) — 분류와 회귀를 모두 다루는 트리 모형

학습 목표

- 의사결정나무의 정의와 동작 원리를 '스무고개' 비유로 설명할 수 있다.
- 뿌리/중간/끝 마디, 분할 기준(splitting rule), 정지 규칙(stopping rule), 끝마디의 예측값 할당 개념을 구분할 수 있다.
- 분류나무(Classification Tree)와 회귀나무(Regression Tree)의 차이를 안다.
- 불순도(impurity) 척도인 지니 지수(Gini index)와 엔트로피(Entropy)의 정의를 알고, 손으로 직접 계산할 수 있다.
- 재귀적 분기(Recursive Partitioning) 알고리즘의 단계를 따라가며 정보획득(Information Gain)이 최대인 분할점을 찾을 수 있다.
- 과적합(Overfitting)이 무엇이며, 가지치기(Pruning)가 왜 필요한지 설명할 수 있다.
- 사이킷런의 DecisionTreeClassifier 로 iris 데이터를 학습하고 plot_tree 로 시각화할 수 있다.
- max_depth, min_samples_split, ccp_alpha 등 주요 하이퍼파라미터의 의미를 설명할 수 있다.
-

1. 의사결정나무란?

의사결정나무(Decision Tree)는 지도학습(Supervised Learning) 알고리즘의 한 종류로, '예/아니오' 형태의 질문을 연속적으로 던져 데이터를 분류하거나 값을 예측하는 모형이다. 사람이 "스무고개"를 하듯 가장 정보가 많은 질문부터 차례로 던져, 답이 한쪽으로 모일 때까지 질문을 세분화해 나간다. 결과적으로 만들어지는 구조가 뿌리에서 가지가 뻗어 나와 잎(leaf)으로 끝나는 '나무' 모양을 하고 있어서 결정 '트리'라는 이름이 붙었다.

의사결정나무는 다음 세 가지 강점 덕분에 입문자에게도, 실무에서도 꾸준히 쓰이고 있다.

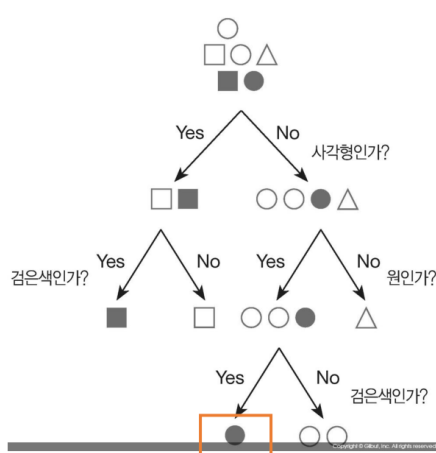
- 결과를 사람이 해석할 수 있다 — 학습된 트리는 if-then 규칙으로 바로 읽을 수 있는 화이트박스(white-box) 모형이다.
- 전처리에 둔감하다 — 단위가 다른 변수를 섞어도, 이상치가 있어도 비교적 안정적으로 작동한다.
- 범주형/수치형 변수를 동시에 받을 수 있다 — Label Encoding 만 해 주면 그대로 학습된다.

1.1 비유: 스무고개와 도형 분류

아래 그림은 같은 도형이라도 '사각형인가?', '원인가?', '검은색인가?' 하는 질문을 차례대로 던지면 특정 도형을 단계별로 좁혀 갈 수 있다는 사실을 보여준다. 각 질문은 단순한 이진(Yes/No) 분기이지만, 질문이 쌓이면 "검은색 원"이라는 구체적인 분류 결과에 도달한다. 의사결정나무도 이와 똑같은 방식으로 각 노드에서 한 변수의 값에 대해 질문을 던지고, 답에 따라 데이터를 둘로 가른다.

의사결정나무(Decision Tree)란?

예시: 다양한 도형을 분류하는 의사 결정 나무



검은색 원이 분류되는 과정

1. 사각형인가? -> No
2. 원인가? -> Yes
3. 검은색인가? -> Yes

그림 1. '사각형 → 원 → 검은색' 세 번의 질문으로 검은색 원이 분류되는 과정

이 흐름에서 한 가지 더 중요한 관찰이 있다. 마지막 질문 "검은색인가?"는 두 번째 질문 "원인가?"에 Yes 라고 답한 데이터에 대해서만 의미가 있다. 즉, 하위 질문은 상위 질문의

답과 연결되어 있고, 이런 위계 구조 덕분에 트리는 변수 간 '조건부' 관계를 자연스럽게 표현한다.

 의사결정나무(Decision Tree)란?

 데이터의 특징에 대한 질문을 하며 응답에 따라 데이터를 분류해가는 알고리즘

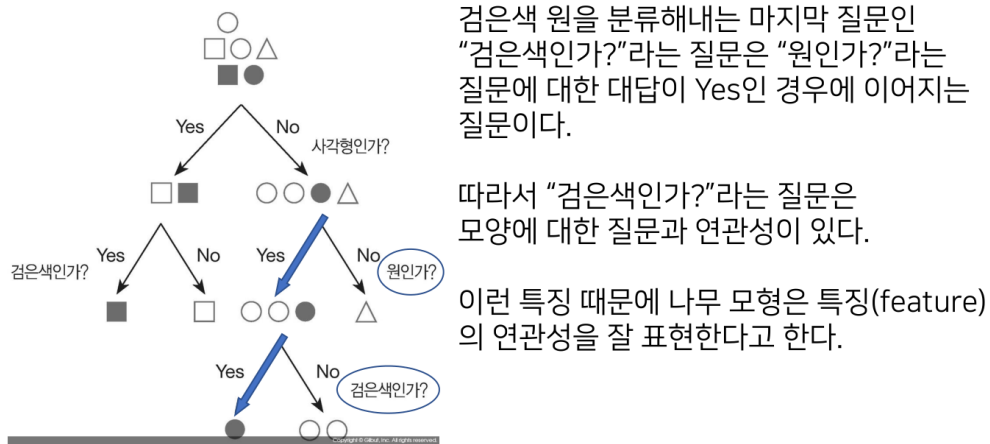


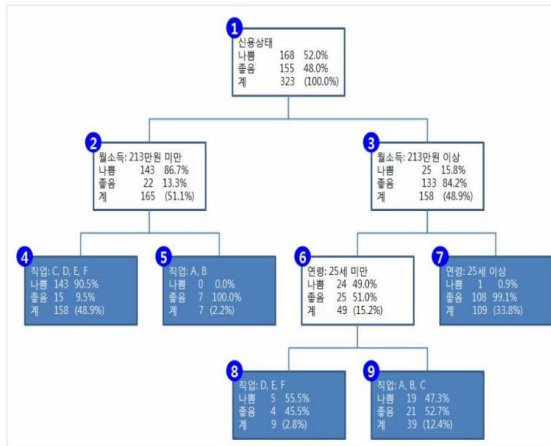
그림 2. 하위 질문은 상위 질문에 의존한다 — 트리는 특징(feature)의 연관성을 잘 표현한다

1.2 의사결정나무의 구성요소

학습이 끝난 의사결정나무는 다음 세 종류의 마디(노드)로 이루어진다.

- 뿌리 마디(Root node): 나무가 시작되는 가장 위쪽 노드. 전체 데이터를 그대로 받는다.
- 중간 마디(Internal node): 분기가 계속되는 도중의 노드. 자식 노드가 있다.
- 끝 마디(Terminal / Leaf node): 더 이상 분기하지 않는 노드. 분류 결과나 예측값이 여기서 결정된다. 분류 규칙의 개수는 끝 마디의 개수와 같다.

주요 특징



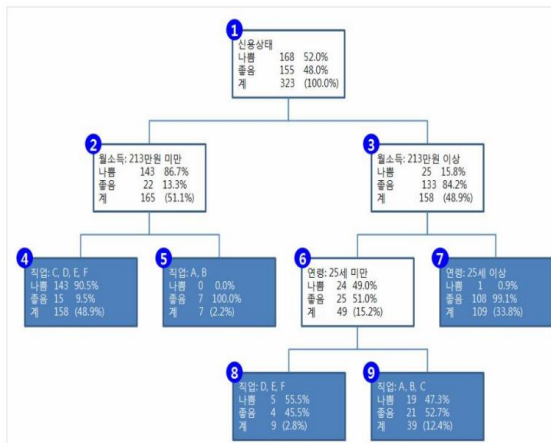
- 뿌리 마디(Root node)
: 나무 구조가 시작되는 노드 ①
- 중간 마디(Intermediate node)
: 끝 마디가 아닌 노드들 ②③⑥
- 끝 마디(Terminal node)
: 각 나무줄기의 끝으로 분류 규칙은 끝나는 노드 개수만큼 생성
④⑤⑦⑧⑨

그림 3. 뿌리 마디·중간 마디·끝 마디로 구성된 의사결정나무 구조

학습 알고리즘이 만들어야 하는 결정은 결국 세 가지로 정리된다.

- 분할 기준(splitting rule) — 어떤 변수의 어떤 값을 기준으로 데이터를 나눌 것인가?
- 정지 규칙(stopping rule) — 언제 분기를 멈추고 끝 마디로 둘 것인가?
- 끝 마디의 예측값 — 끝 마디에 도달한 데이터에 어떤 라벨/값을 할당할 것인가?

주요 특징



- 뿌리마디의 질문이 왜 소득인가?
-> 분할기준(splitting rule)의 선택
- 4번, 5번, 7번 마디들은 끝마디인 반면 6번 마디는 왜 중간마디인가?
-> 정지규칙(stopping rule)
- 7번 마디에 속하는 자료는 신용상태를 어떻게 결정하여야 하는가?
-> 끝마디에서의 예측값 할당법

그림 4. 학습 알고리즘이 결정해야 하는 세 가지 — 분할 기준·정지 규칙·끝마디 예측값

1.3 분류나무 vs 회귀나무

의사결정나무는 종속변수(타겟)의 형태에 따라 두 가지로 나뉜다. 종속변수가 범주형이면 분류나무, 연속형이면 회귀나무다. 둘 다 학습 과정은 동일하게 '재귀적 분기'를 하지만, 끝 마디에서의 예측 방식이 다르다.

구분 ── 분류나무(Classification Tree)

| ── 끝 마디에서 가장 빈도가 높은 범주(최빈값)를 예측값으로 반환

| 예) sunny + 습도>70 → '경기 안 함'

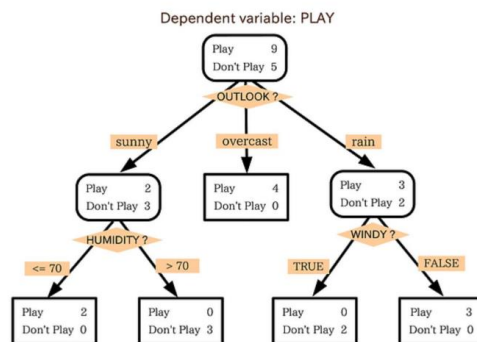
└─ 회귀나무(Regression Tree)

└─ 끝 마디에 속한 데이터의 y 평균을 예측값으로 반환

예) '평수·건축연도가 이 영역인 아파트 평균가 4.3 억'

📌 분류 나무(Classification Tree)

✓ 의사결정나무는 분류(Classification)와 회귀(Regression) 모두 가능



의사결정나무의 분류(Classification)

새로운 데이터가 특정 Terminal node에 속한다는 정보를 확인

↓
해당 Terminal node에서 가장 빈도가 높은 범주에 새로운 데이터를 분류

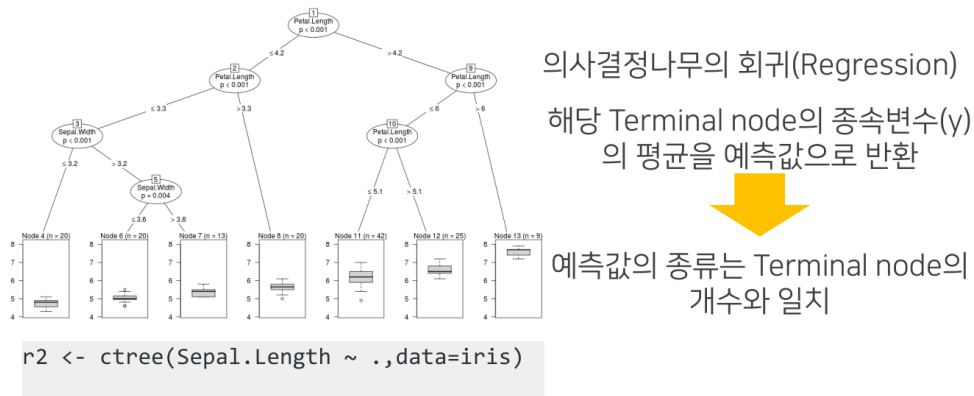
✓ Ex) 날씨는 맑은데(sunny) 습도가 70을 넘는 날(HUMIDITY ? > 70)은 경기가 열리지 않을 것이다.

source : <https://ratsgo.github.io/machine%20learning/2017/03/26/tree/>

그림 5. 분류나무 — 끝 마디에서 가장 흔한 범주를 예측

회귀 나무(Regression Tree)

의사결정나무는 분류(Classification)와 회귀(Regression) 모두 가능



source : <https://stackoverflow.com/questions/29131254/how-to-generate-a-prediction-interval-from-a-regression-tree-rpart-object>

그림 6. 회귀나무 — 끝 마디 데이터의 평균을 예측

2. 활용 사례

의사결정나무는 결과 해석이 명확하다는 장점 덕분에 '규칙을 사람에게 보여줘야 하는' 분야에서 특히 강하다.

- 세분화(Segmentation): 고객/거래/환자 등을 비슷한 집단으로 묶을 때
- 분류(Classification): 신용 등급, 이탈/유지, 양성/음성 같은 라벨 예측
- 예측(Prediction): 시장감시, 부정거래 탐지, 이상 진료 패턴 발견
- 한국거래소 시장감시 시스템 — 의심거래를 트리 규칙으로 자동 분류
- KB 국민카드 스마트오퍼링 — 고객 특성에 맞춰 혜택을 트리 규칙으로 선택

3. 분리 기준 — 무엇을 기준으로 나누는가

좋은 분기는 '분리 후 각 영역이 한 가지 클래스로 모이는' 분기다. 이때 '한 가지 클래스로 모인 정도'를 수치로 표현한 것이 불순도(impurity)이며, 불순도가 낮을수록 순도(purity)는 높다. 의사결정나무는 분기 전후의 불순도 차이(=정보획득, Information Gain)가 가장 큰 분기를 선택한다.

종속변수(반응변수)의 형태에 따라 사용하는 불순도 척도가 달라진다.

반응변수 형태 └ 범주형(Classification)

| └ 카이제곱 통계량의 p 값

| └ 지니 지수(Gini Index) ← CART 의 기본

| └ 엔트로피(Entropy Index) ← C4.5 의 기본

└ 수치형(Regression)

└ 분산 분석의 F 통계량

└ 분산 감소량(Variance Reduction)

✓ 반응 변수가 **범주형**인 경우

☑ 카이제곱 통계량의 p값 : p값이 가장 작은 예측 변수와 그 때의 최적 분리에 의해서 자식 마디를 형성

☑ 지니 지수 : 불순도를 측정하는 하나의 지수로서 지니 지수를 가장 감소시켜주는 예측 변수와 그 때의 최적 분리에 의해서 자식 마디를 선택

☑ 엔트로피 지수 : 이 지수가 가장 작은 예측 변수와 그 때의 최적 분리에 의해서 자식 마디 형성

✓ 반응 변수가 **수치형**인 경우

☑ 분산 분석에서의 F 통계량 : p값이 가장 작은 예측 변수와 그 때의 최적 분리에 의해서 자식 마디 형성

☑ 분산의 감소량 : 예측 오차를 최소화하는 것과 동일한 기준으로 분산의 감소량을 최대화하는 기준의 최적 분리에 의해서 자식 마디 형성

그림 9. 반응변수에 따라 달라지는 분리 기준

3.1 지니 지수 (Gini Index)

지니 지수는 “이 영역에서 무작위로 두 표본을 뽑았을 때, 두 표본의 클래스가 다를 확률”로 해석할 수 있는 불순도 척도다. 한 영역(A)에 m 개의 클래스가 있고, 클래스 k 의 비율이 p_k 라면 지니 지수는 다음과 같다.

$$I(A) = 1 - \sum_k p_k^2$$

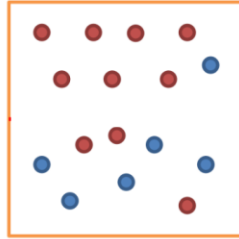
한 영역에 한 가지 클래스만 있으면(예: 모두 ‘빨간색’) $p=1$ 이 되어 $I(A)=0$. 두 클래스가 정확히 반반이면 $I(A)=0.5$ 로 최대다. 즉, 0 에 가까울수록 ‘순수’하다.

지니 지수(Gini Index)의 정의

집합에 이질적인 것이 얼마나 섞였는지를 측정하는 지표로 불순도를 측정한다

$$I(A) = 1 - \sum_{k=1}^m p_k^2$$

✓ p_k = A영역에 속한 레코드 중 k 범주에 속하는 레코드의 비율



예시: 전체 16개 중 빨간색 10개, 파란색 6개

$$I(A) = 1 - \left(\frac{10}{16}\right)^2 - \left(\frac{6}{16}\right)^2 = 0.47$$

그림 10. 빨간색 10, 파란색 6 인 영역의 지니 지수 = $1 - (10/16)^2 - (6/16)^2 = 0.47$

분기 후에 두 개 이상의 영역이 생기면 각 영역의 지니 지수를 '영역 크기 비율 R_i ' 로 가중평균한다.

$$I(A) = \sum_i R_i \cdot (1 - \sum_k p_{ik}^2)$$

예시: 16 개를 위/아래 8 개씩 분할했더니 위가 '빨강 7·파랑 1', 아래가 '빨강 3·파랑 5' 였다면

$$\text{분기 전 } I(A) = 1 - (10/16)^2 - (6/16)^2 = 0.47$$

$$\text{분기 후 } I(A) = (8/16) \cdot (1 - (7/8)^2 - (1/8)^2)$$

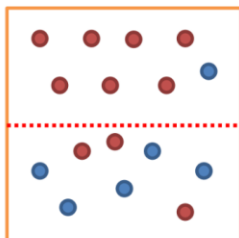
$$+ (8/16) \cdot (1 - (3/8)^2 - (5/8)^2) = 0.34$$

$$\text{정보획득(IG)} = 0.47 - 0.34 = 0.13 \quad \leftarrow \text{클수록 좋은 분기}$$

두 개 이상의 영역에 대한 지니 지수(Gini Index)

$$I(A) = \sum_{i=1}^d (R_i (1 - \sum_{k=1}^m p_{ik}^2))$$

✓ R_i = 분할 전 레코드 중 분할 후 i 영역에 속하는 레코드의 비율



예시: 전체 16개를 8개/8개로 분할

위 : 빨간색 7개/파란색 1개

아래 : 빨간색 3개/파란색 5개

$$\begin{aligned} I(A) &= R_1 \times I(A_1) + R_2 \times I(A_2) \\ &= \left(\frac{8}{16}\right) \times \left(1 - \left(\frac{7}{8}\right)^2 - \left(\frac{1}{8}\right)^2\right) + \left(\frac{8}{16}\right) \times \left(1 - \left(\frac{3}{8}\right)^2 - \left(\frac{5}{8}\right)^2\right) \\ &= 0.34 \end{aligned}$$

(각각에 대해서 지니 계수를 계산하고, 비율 R_i 를 가중치로 고려한다고 생각해도 좋다.)

그림 11. 분기 후 가중평균 지니 지수 계산

아래 코드는 지니 지수를 직접 구현해 위 예시를 그대로 검증한다. (라이브러리 호출 없이도 같은 결과가 나온다.)

```
import numpy as np

def gini(labels):
    """한 영역의 지니 지수."""
    labels = np.asarray(labels)
    if len(labels) == 0:
        return 0.0
    _, counts = np.unique(labels, return_counts=True)
    p = counts / counts.sum()
    return 1.0 - np.sum(p ** 2)

# 분기 전: 빨강 10, 파랑 6
before = ['R']*10 + ['B']*6
print(f'분기 전 Gini = {gini(before):.3f}')    # 0.469

# 분기 후: 위 8개(R7,B1), 아래 8개(R3,B5)
top = ['R']*7 + ['B']*1
bottom = ['R']*3 + ['B']*5
w_top, w_bot = len(top)/16, len(bottom)/16
after = w_top*gini(top) + w_bot*gini(bottom)
print(f'분기 후 Gini = {after:.3f}')          # 0.344
print(f'정보획득(IG) = {gini(before) - after:.3f}')    # 0.125
```

3.2 엔트로피 (Entropy)

엔트로피는 정보이론(information theory)에서 가져온 개념으로, '한 영역의 평균 정보량' 즉 '불확실성'을 의미한다. 역시 0(모두 한 클래스)에서 시작해서 클래스가 골고루 섞일수록 커진다.

$$\text{Entropy}(A) = - \sum_k p_k \cdot \log_2(p_k)$$

$$\text{Entropy}(A) = \sum_i R_i \cdot (- \sum_k p_{ik} \cdot \log_2(p_{ik})) \quad \leftarrow \text{분기 후}$$

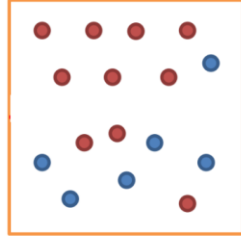
같은 16개 데이터에 대해 엔트로피를 계산해 보면 분기 전은 약 0.95, 위/아래로 가른 뒤는 약 0.75로, 0.20만큼의 정보획득이 발생한다.

🔍 엔트로피 지수(Entropy Index)의 정의

📌 집합에 이질적인 것이 얼마나 섞였는지를 측정하는 지표로 불순도를 측정한다.

$$Entropy(A) = - \sum_{k=1}^m p_k \log_2(p_k)$$

- ✓ m개의 레코드가 속하는 A영역에 대한 엔트로피
- ✓ p_k = A영역에 속한 레코드 중 k 범주에 속하는 레코드의 수



$$\begin{aligned} Entropy(A) &= - \frac{10}{16} \log_2\left(\frac{10}{16}\right) - \frac{6}{16} \log_2\left(\frac{6}{16}\right) \\ &= 0.95 \end{aligned}$$

- ✓ 모든 레코드가 동일한 범주에 속할 경우 $Entropy(A) = 0$

그림 12. 한 영역의 엔트로피 계산 — 모두 한 클래스면 0

```
import numpy as np

def entropy(labels):
    labels = np.asarray(labels)
    if len(labels) == 0:
        return 0.0
    _, counts = np.unique(labels, return_counts=True)
    p = counts / counts.sum()
    return -np.sum(p * np.log2(p))

before = ['R']*10 + ['B']*6
top     = ['R']*7 + ['B']*1
bottom = ['R']*3 + ['B']*5
after  = 0.5*entropy(top) + 0.5*entropy(bottom)

print(f'분기 전 Entropy = {entropy(before):.3f}')    # 0.954
print(f'분기 후 Entropy = {after:.3f}')              # 0.751
print(f'정보획득(IG)    = {entropy(before)-after:.3f}') # 0.203
```

3.3 지니 vs 엔트로피 — 어떤 것을 쓸까

두 지표 모두 '0에 가까울수록 순수'라는 동일한 방향성을 가진다. 실제로 두 지표의 모양을 그려 보면 거의 같은 형태이고, 같은 분기점을 선택하는 경우가 대부분이다. 다만 지니는 로그 연산이 없어 계산이 빠르고, 엔트로피는 정보이론에 기반한 직관이 명확하다.

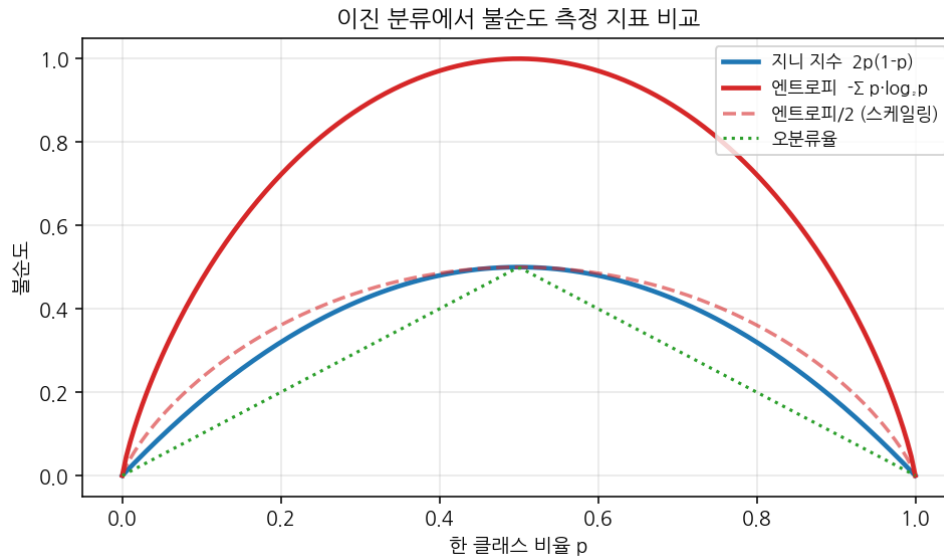


그림 13. 이진 분류에서 한 클래스 비율 p 에 따른 지니·엔트로피·오분류율

정리

- └ 지니: 0 ~ 0.5 (이진 분류 기준 최댓값 0.5)
 - └ 사이킷런 DecisionTreeClassifier 의 기본값 criterion='gini'
- └ 엔트로피: 0 ~ 1
 - └ criterion='entropy' 로 지정하면 사용
- └ 결론: 보통 어느 쪽을 써도 결과는 유사 - 속도 우선이면 지니, 정보이론적 해석이 필요하면 엔트로피

4. 의사결정나무의 학습 과정

의사결정나무의 학습은 크게 두 단계로 나뉜다. 먼저 순도가 높아지는 방향으로 데이터를 계속 쪼개 가는 재귀적 분기(Recursive Partitioning) 단계가 있고, 그 다음 너무 잘게 쪼개진 나무를 다시 합치는 가지치기(Pruning) 단계가 이어진다.

4.1 재귀적 분기 (Recursive Partitioning)

재귀적 분기란 "한 변수의 가능한 모든 분리점에 대해 지니/엔트로피 감소량을 계산하고, 정보획득이 가장 큰 분리점을 선택한 뒤, 그 결과로 생긴 두 영역에 대해 같은 작업을 반복"하는 절차다. 분기 횟수를 미리 정해 두지 않고 정지 조건을 만날 때까지 '재귀적으로' 반복한다는 데에서 이름이 붙었다.

재귀적 분기(Recursive Partitioning) 방법

- 특정 영역에 속하는 개체들을 하나의 기준 변수 값의 범위에 따라 분기한다.
- 새로 생성될 자식 노드의 동질성이 최대화되도록 분기점을 선택한다.
- 불순도(Impurity)를 측정하는 기준으로는 범주형 변수에 대해 지니계수, 수치형 변수에 대해 분산을 이용한다.
- 분기 횟수가 정해지지 않은 채로 사전에 설정한 기준을 만족할 때까지 분기를 반복하는 데서 재귀적 분기라는 이름이 붙었다.

그림 14. 재귀적 분기 — 동질성이 최대가 되는 분기점을 변수별로 탐색

교과서적인 예시로 잔디깎기 기계(Owner / Non-owner) 구매 여부를 24 가구의 소득과 주택 크기로 분류하는 문제가 자주 쓰인다. Owner 12 명·Non-owner 12 명으로 시작해 분기 전 지니 지수는 $1 - (12/24)^2 - (12/24)^2 = 0.5$ 이다.

재귀적 분기(Recursive Partitioning) 방법 예

- 24개 가정을 대상으로 소득(Income), 주택크기(Lot size) 잔디깎기 기계 구입 여부(Ownership)를 조사한 데이터
- Owner 12명, Non-owner 12명

Income	Lot size	Ownership	Income	Lot size	Ownership
60.0	18.4	Owner	75.0	19.6	Non-owner
85.5	16.8	Owner	52.8	20.8	Non-owner
64.8	21.6	Owner	64.8	17.2	Non-owner
61.5	20.8	Owner	43.2	20.4	Non-owner
87.0	23.6	Owner	84.0	17.6	Non-owner
110.1	19.2	Owner	49.2	17.6	Non-owner
108.0	17.6	Owner	59.4	16.0	Non-owner
82.8	22.4	Owner	66.0	18.4	Non-owner
69.0	20.0	Owner	47.4	16.4	Non-owner
93.0	20.8	Owner	33.0	18.8	Non-owner
51.0	22.0	Owner	51.0	14.0	Non-owner
81.0	20.0	Owner	63.0	14.8	Non-owner

source : <https://ratsgo.github.io/machine%20learning/2017/03/26/tree/>

그림 15. 24 가구의 소득(Income)·주택크기(Lot size)·구매여부(Ownership) 데이터

이제 한 변수(예: Lot size)를 오름차순 정렬한 뒤, 인접한 두 값의 중간을 가능한 모든 분리점으로 두고 각 분리점에서 분기 후 지니 지수를 계산한다. 정보획득(분기 전 - 분기 후)이 최대인 분리점을 그 변수의 '최적 분리점'으로 선택한다.

재귀적 분기(Recursive Partitioning) 방법 예

	income	lotsize	ownership
1	51.0	14.0	non-owner
2	63.0	14.8	non-owner
3	59.4	16.0	non-owner
4	47.4	16.4	non-owner
5	85.5	16.8	owner
6	64.8	17.2	non-owner
7	108.0	17.6	owner
8	84.0	17.6	non-owner
9	49.2	17.6	non-owner
10	60.0	18.4	owner
11	66.0	18.4	non-owner
12	33.0	18.8	non-owner
13	110.1	19.2	owner
14	75.0	19.6	non-owner
15	69.0	20.0	owner
16	81.0	20.0	owner
17	43.2	20.4	non-owner
18	61.5	20.8	owner
19	93.0	20.8	owner
20	52.8	20.8	non-owner
21	64.8	21.6	owner
22	51.0	22.0	owner
23	82.8	22.4	owner
24	87.0	23.6	owner

1. 한 변수 기준으로 정렬 ex) Lot size를 오름차순으로 정렬

2. 가능한 모든 분기점에 대해 지니 지수를 구해 분할 전과 비교해 정보획득을 조사한다.

2-1. 분기지점을 첫 레코드와 나머지 23개 레코드로 설정

1번 레코드와 2~24번 레코드 간의 지니 지수를 구한 뒤 이를 분기 전 지니 지수와 비교해 정보획득을 조사한다.

분기 전 지니 지수 = $1 - \left(\frac{12}{24}\right)^2 - \left(\frac{12}{24}\right)^2 = 0.5$

$$\text{분기 후 지니 지수} = \frac{1}{24} \left(1 - \left(\frac{0}{1}\right)^2 - \left(\frac{1}{1}\right)^2\right) + \frac{23}{24} \left(1 - \left(\frac{12}{23}\right)^2 - \left(\frac{11}{23}\right)^2\right) = 0.478$$

$$\text{정보획득} = 0.5 - 0.478 = 0.022$$

그림 16. 1 번 분리점 — 1 번 레코드만 따로 두면 정보획득 0.022

재귀적 분기(Recursive Partitioning) 방법 예

	income	lotsize	ownership
1	51.0	14.0	non-owner
2	63.0	14.8	non-owner
3	59.4	16.0	non-owner
4	47.4	16.4	non-owner
5	85.5	16.8	owner
6	64.8	17.2	non-owner
7	108.0	17.6	owner
8	84.0	17.6	non-owner
9	49.2	17.6	non-owner
10	60.0	18.4	owner
11	66.0	18.4	non-owner
12	33.0	18.8	non-owner
13	110.1	19.2	owner
14	75.0	19.6	non-owner
15	69.0	20.0	owner
16	81.0	20.0	owner
17	43.2	20.4	non-owner
18	61.5	20.8	owner
19	93.0	20.8	owner
20	52.8	20.8	non-owner
21	64.8	21.6	owner
22	51.0	22.0	owner
23	82.8	22.4	owner
24	87.0	23.6	owner

2-2. 이후 분기 지점을 두번째 레코드로 두고 처음 두 개 레코드와 나머지 22개 레코드 간의 지니 지수를 계산한 뒤 정보획득을 알아본다.

분기지점을 첫 두 개의 레코드와 나머지 22개 레코드로 설정

1~2번 레코드와 3~24번 레코드 간의 지니 지수를 구한 뒤 이를 분기 전 지니 지수와 비교해 정보획득을 조사한다.

분기 전 지니 지수 = $1 - \left(\frac{12}{24}\right)^2 - \left(\frac{12}{24}\right)^2 = 0.5$

$$\text{분기 후 지니 지수} = \frac{1}{24} \left(1 - \left(\frac{0}{2}\right)^2 - \left(\frac{2}{2}\right)^2\right) + \frac{22}{24} \left(1 - \left(\frac{12}{22}\right)^2 - \left(\frac{10}{22}\right)^2\right) = 0.455$$

$$\text{정보획득} = 0.5 - 0.455 = 0.045$$

그림 17. 2 번 분리점 — 1~2 번을 따로 두면 정보획득 0.045

같은 과정을 23 번째 분리점까지 반복해 보면, 14 번째 분리점에서 정보획득이 최대가 된다. 이때 분리값은 인접한 두 lotsize 19.6 과 20.0 의 중간인 19.8 이다.

재귀적 분기(Recursive Partitioning) 방법 예

2-3. 이후 분기 지점을 세 번째, 네 번째, ..., 23번째까지 두고 지니 지수를 계산해서 정보 획득이 최대가 되는 분기 지점을 찾는다.

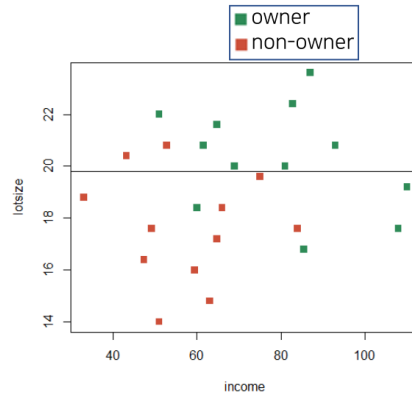
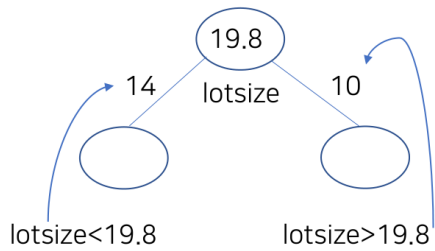


그림 18. lotsize = 19.8 을 기준으로 24 개가 14 개·10 개로 분리

나머지 변수(income)에 대해서도 동일한 작업을 수행해 두 변수 중 더 좋은 분리를 채택한 뒤, 그렇게 만들어진 두 자식 노드 각각에서 다시 재귀적으로 분기한다. 모든 끝 마디의 순도가 100% 가 되면 분기를 멈추는데, 이런 트리를 풀 트리(Full Tree)라고 한다.

재귀적 분기(Recursive Partitioning) 방법 예

3. 다른 변수인 소득을 기준으로 정렬하고 다시 같은 작업을 반복
(순도가 100%이면 분류 끝)

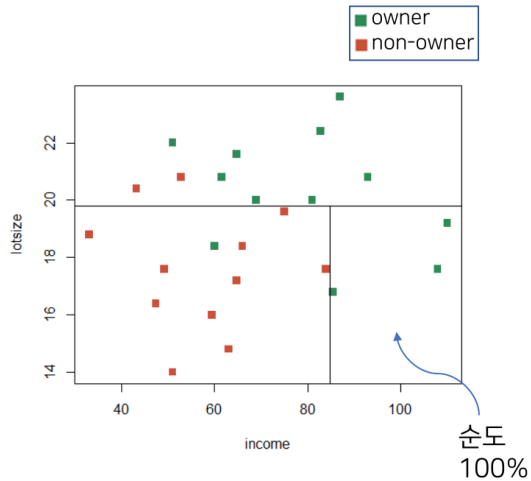
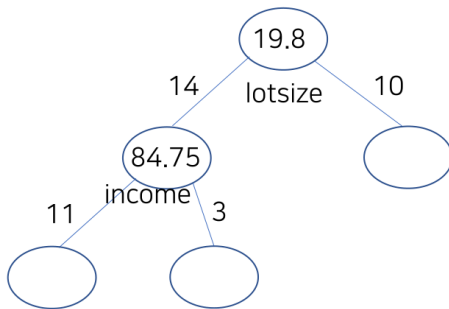


그림 19. 재귀 분기를 반복해 만들어진 풀 트리의 일부

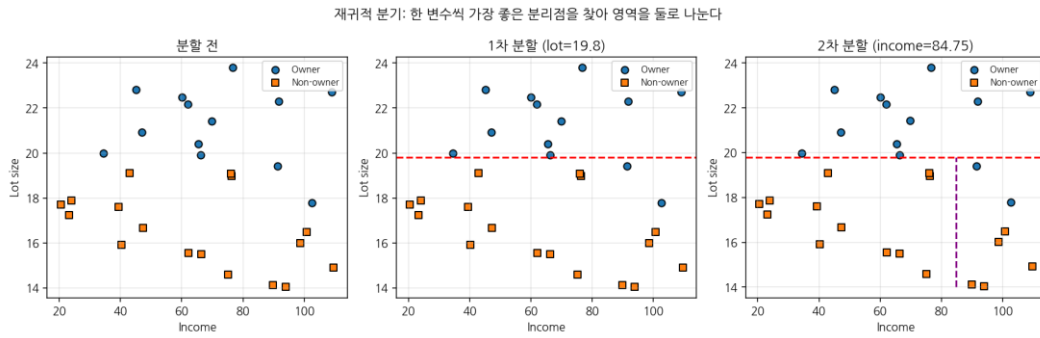


그림 20. 재귀적 분기 — 한 변수씩 영역을 둘로 잘라 가며 동질성을 끌어올린다

4.2 가지치기 (Pruning)

풀 트리는 훈련 데이터에 대해 100%의 정확도를 보이지만, 검증/테스트 데이터에서는 오히려 성능이 떨어지는 경우가 많다. 이를 과적합(Overfitting)이라고 한다. 가지치기는 “ 지나치게 잘게 쪼개진 가지를 다시 합치는” 작업으로, 데이터를 버리는 것이 아니라 분기를 합쳐 트리를 단순화하는 것이다.

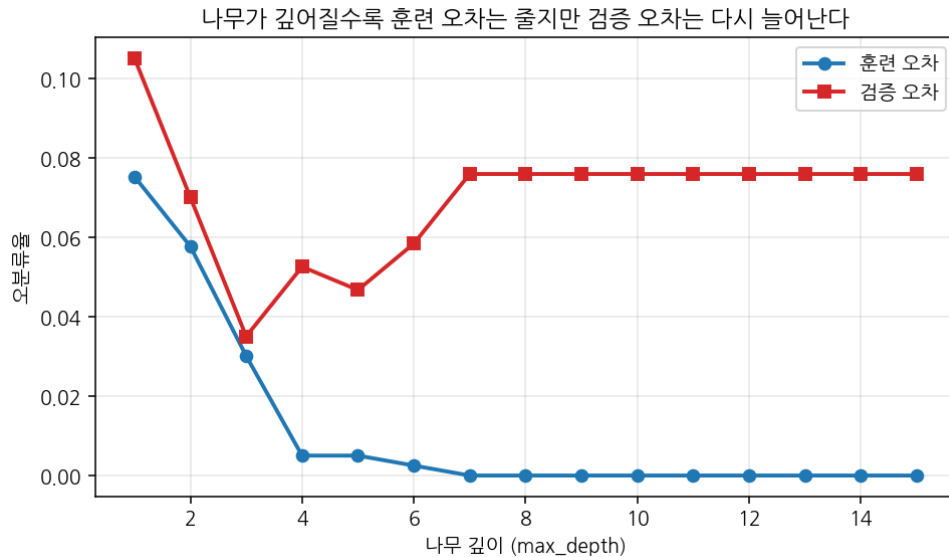
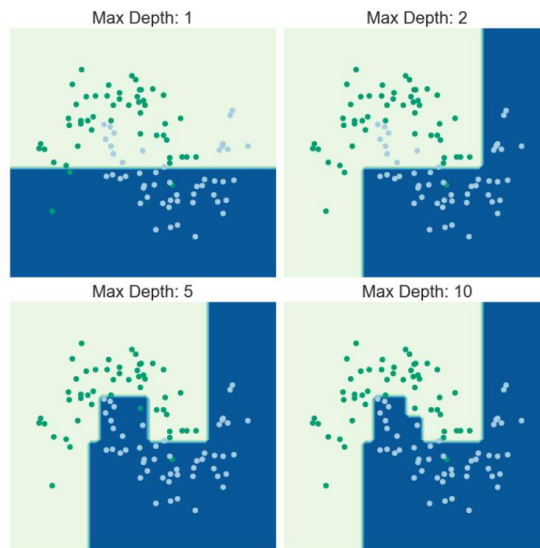


그림 21. 깊어질수록 훈련 오차는 줄지만 검증 오차는 다시 늘어난다(과적합)



source : <http://engineering.pivotal.io/post/interpreting-decision-trees-and-random-forests/>

그림 22. 분기 수와 검증 오분류율 — 적절한 시점에 가지를 쳐야 한다

가지치기는 시점에 따라 두 가지로 구분한다.

- 사전 가지치기(Pre-pruning) — 트리를 만드는 동안 '이 노드를 더 쪼개도 충분한 이익이 없다' 싶으면 그 자리에서 멈춘다. max_depth, min_samples_split 같은 하이퍼파라미터로 제어한다.
- 사후 가지치기(Post-pruning) — 일단 풀 트리까지 만들어 둔 뒤, 검증 데이터의 오류와 트리의 복잡도를 함께 보면서 가지를 잘라낸다. 사이킷런의 cost-complexity 가지치기(ccp_alpha)가 대표적이다.

🔍 가지치기(Pruning) 방법 (2/2)

- 📌 과적합(Overfitting)을 방지하기 위하여 하위 노드들을 상위 노드로 결합한다.
- 📌 Pre-pruning : Tree를 생성하는 과정에서 최소 분기 기준을 이용하는 사전적 가지치기
- 📌 Post-pruning : Full-tree 생성 후, 검증 데이터의 오분류율과 Tree의 복잡도(끝 노드의 수)등을 고려하는 사후적 가지치기

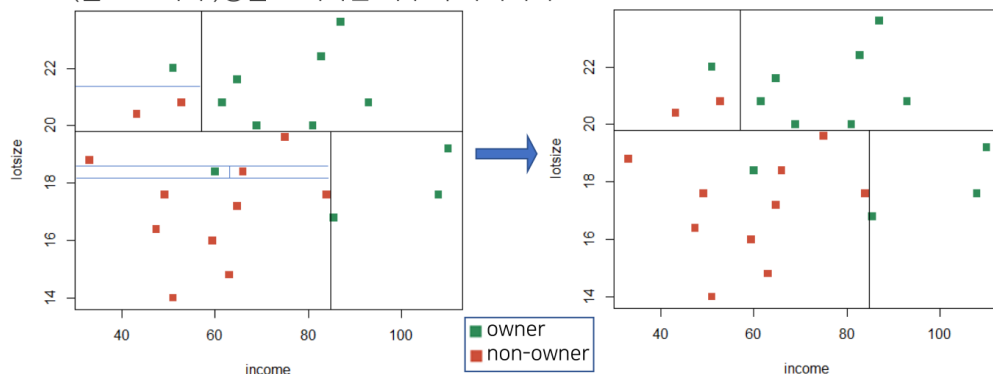


그림 23. 가지치기 후 트리 — 끝 마디 수가 줄어 모형이 단순해진다

사후 가지치기의 비용함수는 다음과 같다. 정확도(오차의 적음)와 단순성(끝마디 수의 적음)을 동시에 추구한다.

$$C\alpha(T) = \text{Err}(T) + \alpha \cdot L(T)$$

- $\text{Err}(T)$: 검증 데이터에 대한 오분류율
- $L(T)$: 끝 마디(terminal node) 수 $\rightarrow$ 모형 복잡도
- α : 두 항을 결합하는 complexity parameter (하이퍼파라미터, 보통 0.01 ~ 0.1)

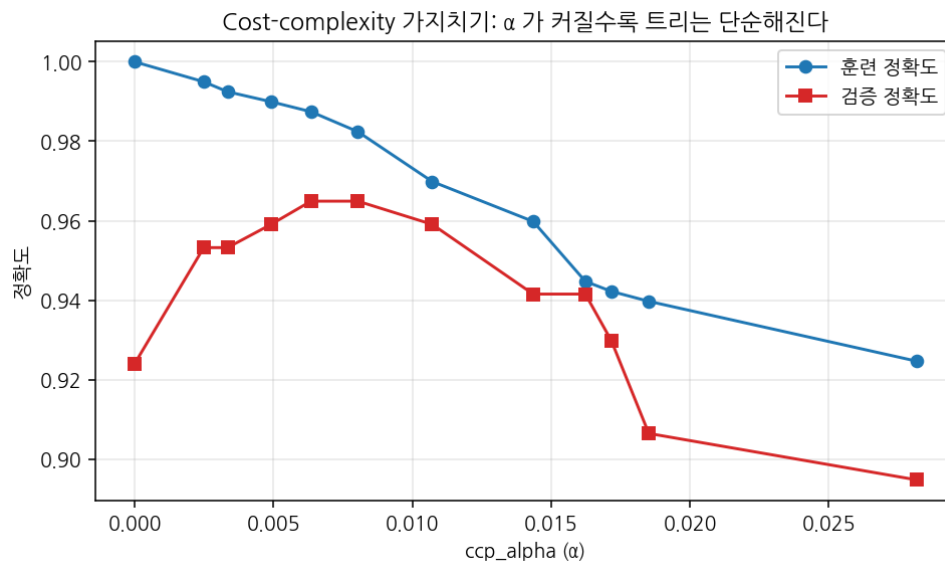


그림 24. ccp_alpha 가 커질수록 트리는 단순해지고 일정 구간에서 검증 정확도가 가장 높다

5. 사이킷런 코드 실습 — iris 데이터로 분류나무 만들기

앞에서 배운 개념을 사이킷런의 DecisionTreeClassifier 로 직접 확인해 보자. 데이터는 손에 익은 iris(붓꽃) 데이터를 쓴다. 4 개의 수치형 특성으로 3 종 붓꽃을 분류한다.

실습 환경 준비

```
# 필요한 모듈
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.tree import DecisionTreeClassifier, plot_tree, export_text
from sklearn.metrics import accuracy_score, classification_report, confusion_matrix
```

[1 단계] 데이터 로드와 살펴보기

iris 데이터는 4 개 특성(sepal length, sepal width, petal length, petal width)과 타겟(setosa=0, versicolor=1, virginica=2)으로 구성된 150 개의 표본이다. DataFrame 으로 옮겨두면 describe() 같은 pandas 기능을 그대로 쓸 수 있다.

```
iris = load_iris()
X = pd.DataFrame(iris.data, columns=iris.feature_names)
y = pd.Series(iris.target, name='species')

print(X.shape)          # (150, 4)
print(y.value_counts())  # 0,1,2 각 50 개 - 클래스 균형
X.describe()            # 단위/스케일 차이 확인
```

[2 단계] 학습 / 테스트 분리

의사결정나무는 학습 데이터에 쉽게 과적합되기 때문에 평가 데이터를 미리 떼어두는 것이 매우 중요하다. stratify=y 를 주면 클래스 비율이 두 세트에서 동일하게 유지된다.

```
X_train, X_test, y_train, y_test = train_test_split(
    X, y,
    test_size=0.3,          # 30% 를 검증용
    stratify=y,             # 클래스 균형 유지
    random_state=42,
)
print(X_train.shape, X_test.shape)  # (105, 4) (45, 4)
```

[3 단계] DecisionTreeClassifier 학습

scikit-learn 의 DecisionTreeClassifier 는 기본적으로 CART 알고리즘에 지니 지수를 사용한다. max_depth=3 으로 비교적 얇은 트리를 학습시켜 보자. 깊이가 얇은 트리는 해석하기에도 좋다.

```
clf = DecisionTreeClassifier(
    criterion='gini',      # 'entropy' 도 가능
    max_depth=3,          # 사전 가지치기
    random_state=0,
)
clf.fit(X_train, y_train)

train_acc = clf.score(X_train, y_train)
test_acc = clf.score(X_test, y_test)
print(f'train accuracy = {train_acc:.3f}')
print(f'test accuracy = {test_acc:.3f}')
```

[4 단계] 트리 시각화 — 학습된 규칙 직접 보기

의사결정나무의 가장 큰 매력은 학습 결과를 그림으로 볼 수 있다는 점이다. plot_tree 는 각 노드에서 사용한 변수, 분리 임계값, 그 노드의 지니 지수와 샘플 개수, 클래스별 분포를 함께 보여준다.

```
fig, ax = plt.subplots(figsize=(11, 6))
plot_tree(
    clf,
    feature_names=iris.feature_names,
    class_names=iris.target_names,
    filled=True, rounded=True, fontsize=9, ax=ax,
)
plt.show()
```

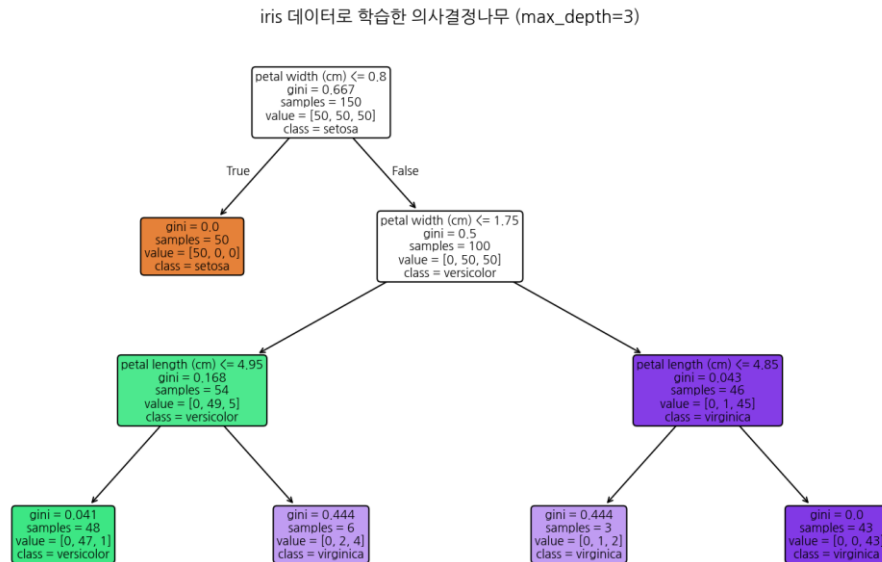


그림 25. iris 로 학습한 의사결정나무 (max_depth=3) — petal width/length 만 사용해도 잘 분류된다
문자로 규칙을 보고 싶다면 export_text 가 편하다.

```
print(export_text(clf, feature_names=iris.feature_names))
# |--- petal width (cm) <= 0.80
# | |--- class: setosa
# |--- petal width (cm) > 0.80
# | |--- petal length (cm) <= 4.75
# ...
```

[5 단계] 결정 경계 살펴보기

의사결정나무는 각 분기가 '축에 수직인 직선' 이므로 결정 경계가 항상 계단(스텝) 모양이 된다. max_depth 가 커질수록 경계가 잘게 쪼개진다.

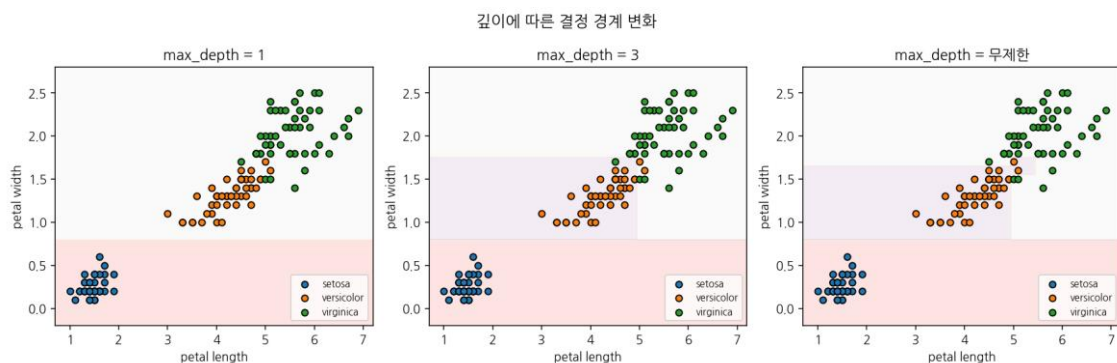


그림 26. max_depth=1, 3, 무제한 — 깊이가 깊어질수록 경계가 잘게 쪼개진다

[6 단계] 평가 — 혼동행렬과 분류 리포트

```
y_pred = clf.predict(X_test)
print(confusion_matrix(y_test, y_pred))
print(classification_report(y_test, y_pred, target_names=iris.target_names))
```

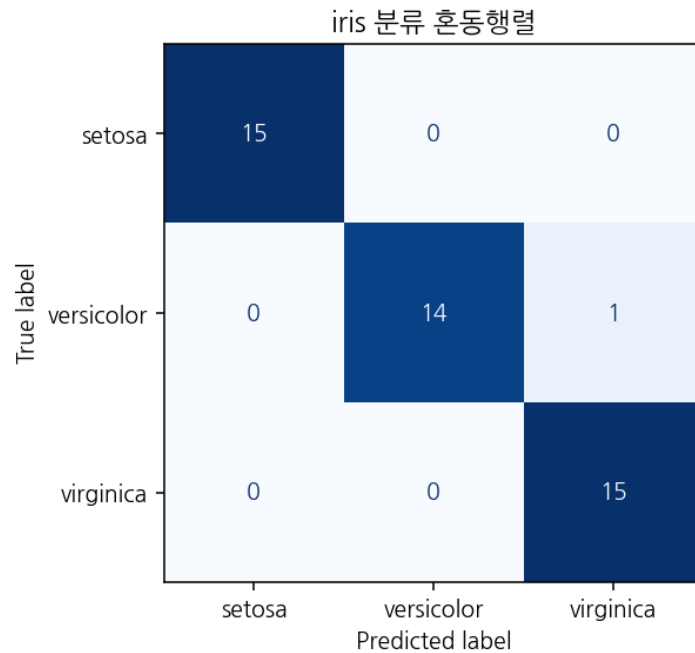


그림 27. iris 분류 혼동행렬 — 대각선이 진할수록 잘 맞춘 것

[7 단계] 특성 중요도(feature_importances_) 확인

학습된 트리가 어떤 특성을 더 많이 활용했는지는 feature_importances_ 로 확인한다. 이 값은 각 특성이 만든 '불순도 감소량의 합'을 정규화한 것이다.

```
import pandas as pd
fi = pd.Series(clf.feature_importances_, index=iris.feature_names)
print(fi.sort_values(ascending=False))
fi.sort_values().plot.barh()
plt.title('feature importances'); plt.show()
```

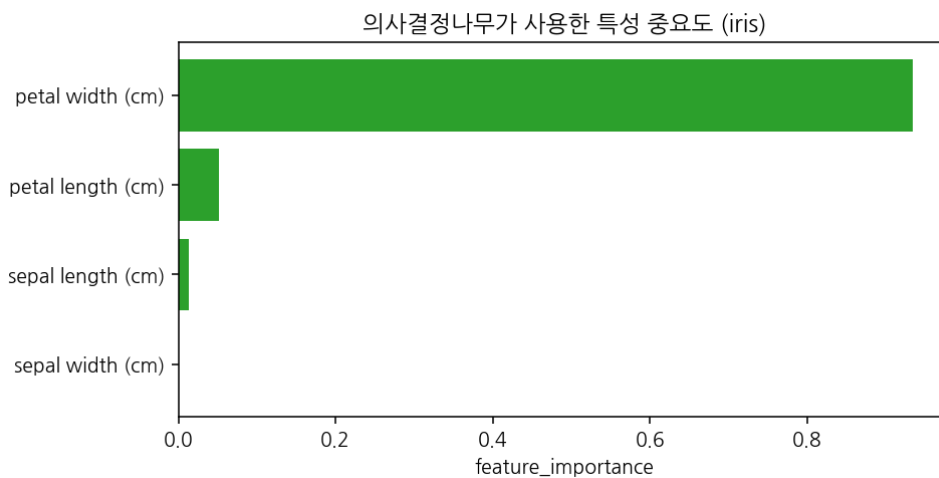


그림 28. iris 트리의 특성 중요도 — petal 관련 두 변수가 거의 모든 분기를 담당

[8 단계] 주요 하이퍼파라미터 — 사전 가지치기

DecisionTreeClassifier 에는 과적합을 통제할 수 있는 여러 사전 가지치기 옵션이 있다. 각각 '분기가 일어날 조건을 까다롭게 만드는' 방향이라 트리가 너무 깊어지는 것을 막아 준다.

주요 하이퍼파라미터

- └─ max_depth : 트리의 최대 깊이. 가장 직관적이고 강력한 제약
- └─ min_samples_split : 분기하려면 노드에 최소 몇 개 표본이 있어야 하는가
- └─ min_samples_leaf : 끝 마디에 최소 몇 개 표본이 남아야 하는가
- └─ max_leaf_nodes : 끝 마디의 최대 개수
- └─ criterion : 'gini' | 'entropy' | 'log_loss'
- └─ ccp_alpha : 사후 가지치기(cost-complexity pruning) 강도 α

```
for d in [1, 2, 3, 5, None]:
    m = DecisionTreeClassifier(max_depth=d, random_state=0).fit(X_train, y_train)
    print(f'max_depth={d}: train={m.score(X_train,y_train):.3f}, '
          f'test={m.score(X_test,y_test):.3f}')
# max_depth=1 : train=0.667, test=0.667
# max_depth=2 : train=0.962, test=0.933
# max_depth=3 : train=0.971, test=0.956
# max_depth=5 : train=1.000, test=0.956
# max_depth=None : train=1.000, test=0.956 ← 풀 트리(과적합 가능성)
```

[9 단계] 사후 가지치기 — ccp_alpha

일단 풀 트리를 만든 뒤 α 값을 조금씩 키워 가며 가지치기를 시도한다.

cost_complexity_pruning_path 가 가능한 α 후보들을 돌려주므로 그중에서 검증 정확도가 가장 좋은 값을 고른다.

```
clf_full = DecisionTreeClassifier(random_state=0).fit(X_train, y_train)
path = clf_full.cost_complexity_pruning_path(X_train, y_train)

best_alpha, best_acc = 0.0, 0.0
for a in path.ccp_alphas[:-1]:
    m = DecisionTreeClassifier(random_state=0, ccp_alpha=a).fit(X_train, y_train)
    acc = m.score(X_test, y_test)
    if acc > best_acc:
        best_alpha, best_acc = a, acc
print(f'best ccp_alpha = {best_alpha:.4f}, test acc = {best_acc:.3f}')
```

6. 의사결정나무의 장단점

6.1 장점

- 화이트박스(white-box) 모형이라 결과를 사람이 보고 해석하기 쉽다.
- 스케일링·정규화 등 별도 전처리가 거의 필요 없다.

- 수치형과 범주형 변수를 함께 다룰 수 있다.
- 이상치(outlier)에 비교적 강하다 — 한쪽 끝 마디로 모일 뿐 모형 전체가 흔들리지 않는다.
- 대규모 데이터에서도 학습 속도가 빠르다.

6.2 단점

- 탐욕적(greedy) 알고리즘이라 전역적으로 최적인 트리를 보장하지 못한다.
- 깊게 자라면 쉽게 과적합된다 — 가지치기 또는 앙상블(랜덤포레스트, 부스팅) 로 보완해야 한다.
- 결정 경계가 '축에 수직인 계단' 형태라서, 비스듬한 경계를 가진 데이터에는 트리가 매우 복잡해진다.
- 작은 데이터 변화에도 트리의 모양이 크게 달라지는 '분산이 큰' 알고리즘이다 — 그래서 단일 트리보다 랜덤포레스트가 자주 쓰인다.

오늘 수업 정리

- 의사결정나무는 if-then 규칙을 '재귀적 분기'로 학습하는 화이트박스 모형이다.
- 분기의 기준은 불순도 감소(=정보획득)다. 분류에서는 지니 지수와 엔트로피, 회귀에서는 분산 감소량을 쓴다.
- 풀 트리는 과적합 위험이 크므로 max_depth, min_samples_split, ccp_alpha 등으로 가지치기를 한다.
- 사이킷런의 DecisionTreeClassifier + plot_tree 로 학습된 규칙을 그대로 시각화할 수 있다.
- 단일 트리의 한계(분산 큼)는 다음 차시에 다룰 랜덤포레스트와 부스팅에서 본격적으로 보완된다.

과제

- breast_cancer 데이터에 DecisionTreeClassifier 를 학습시키고, max_depth 를 1~10 으로 바꾸어 가며 훈련/검증 정확도 곡선을 그리시오.
- 같은 데이터에 cost_complexity_pruning_path 로 ccp_alpha 후보를 뽑은 뒤, 가장 좋은 α 를 선택하고 가지치기 전후 트리의 끝마디 수를 비교하시오.

- `criterion='gini'` 와 `criterion='entropy'` 를 각각 학습시켜, 두 트리의 구조와 정확도를 비교한 짧은 노트(3 줄 이상)를 작성하시오.